PSEUDOCODE, ALGORITHM DESIGN, AND TIME COMPLEXITY ANALYSIS

1. Commit Operation

The commit operation is used to save changes into the currently active branch. A new commit node is created and inserted at the end of the branch linked list.

Algorithm Steps

1. Start the operation.
2. Check whether the commit message is empty.
3. If the message is empty:
    - Display an error message.
    - Stop the operation.
4. Generate a unique commit ID.
5. Obtain the current system date and time.
6. Create a new commit node containing:
    - Commit ID
    - Commit message
    - Timestamp
7. Insert the new node at the end of the linked list.
8. Update the tail pointer to the new node.
9. Increase the branch commit count.
10. Display success message.
11. End the operation.

Pseudocode

```
ALGORITHM Commit(message)

BEGIN

    IF message is empty THEN
        DISPLAY "Commit message cannot be empty"
        RETURN
    END IF

    id ← GenerateCommitID()

    timestamp ← GetCurrentTime()

    Create new CommitNode

    newNode.id ← id
    newNode.message ← message
    newNode.timestamp ← timestamp

    Append newNode at end of current branch list
```

```
    Update tail pointer

    Increase branch size

    DISPLAY "Commit added successfully"

END
```

Time Complexity

T(n)=O(1)

The commit operation inserts a node directly at the end of the linked list using the tail pointer. Since traversal is not required, the execution time remains constant regardless of the number of commits.

2. Create Branch Operation

The create branch operation generates a new branch by copying all commits from the currently active branch.

Algorithm Steps

1. Start the operation.
2. Check whether the branch name is empty.
3. If empty:
    - Display error message.
    - Stop operation.
4. Check whether the branch already exists.
5. If branch exists:
    - Display error message.
    - Stop operation.
6. Create a new empty linked list.
7. Traverse all commits in the current branch.
8. Copy each commit into the new branch.
9. Store the new branch inside the branch table.
10. Display success message.
11. End operation.

Pseudocode

```
ALGORITHM CreateBranch(branchName)

BEGIN

    IF branchName is empty THEN
        DISPLAY "Invalid branch name"
```

```
        RETURN
    END IF

    IF branch already exists THEN
        DISPLAY "Branch already exists"
        RETURN
    END IF

    Create new empty branch

    FOR each commit in current branch DO

        Copy commit into new branch

    END FOR

    Store new branch in branch table

    DISPLAY "Branch created successfully"

END
```

Time Complexity

T(n)=O(n)

Complexity Explanation

The algorithm copies every commit from the current branch into the new branch. Therefore, the execution time depends on the total number of commits.

3. Switch Branch Operation

The switch branch operation changes the active working branch.

Algorithm Steps

1. Start operation.
2. Check whether the requested branch exists.
3. If branch does not exist:
    - Display error message.
    - Stop operation.
4. Update current branch pointer.
5. Display success message.
6. End operation.

Pseudocode

```
ALGORITHM SwitchBranch(branchName)

BEGIN

    IF branch does not exist THEN
        DISPLAY "Branch not found"
        RETURN
    END IF

    currentBranch ← branchName

    DISPLAY "Branch switched successfully"

END
```

Time Complexity

T(n)=O(1)

The branch is accessed directly using a hash map (`unordered_map`), making the operation very fast.

4. Enqueue Merge Request Operation

The enqueue merge operation stores merge requests in FIFO order using a queue.

Algorithm Steps

1. Start operation.
2. Check whether source branch exists.
3. If branch does not exist:
    - Display error message.
    - Stop operation.
4. Check whether source branch equals target branch.
5. If both are same:
    - Display error message.
    - Stop operation.
6. Create merge request node.
7. Insert node at rear of queue.
8. Update rear pointer.
9. Increase queue size.
10. Display success message.
11. End operation.

Pseudocode

```
ALGORITHM EnqueueMerge(sourceBranch)

BEGIN

    IF sourceBranch does not exist THEN
        DISPLAY "Source branch not found"
        RETURN
    END IF

    IF sourceBranch = currentBranch THEN
        DISPLAY "Cannot merge branch into itself"
        RETURN
    END IF

    Create MergeRequest node

    Insert node at rear of queue

    Update rear pointer

    Increase queue size

    DISPLAY "Merge request added successfully"

END
```

Time Complexity

T(n)=O(1)

The merge request is directly inserted at the rear of the queue without traversal.

5. Process Merge Request Operation

Processes merge requests in FIFO order and applies unique commits from the source branch into the target branch.

Algorithm Steps

1. Start operation.
2. Check whether merge queue is empty.
3. If empty:
    - Display message.
    - Stop operation.
4. Remove first merge request from queue.
5. Validate source and target branches.
6. Create set to store target commit IDs.
7. Traverse target branch and insert all IDs into set.

8. Traverse source branch commits.
9. For each commit:
    - Check whether commit already exists in target branch.
    - If not present:
        - Add commit into temporary queue.
10. Process temporary queue in FIFO order.
11. Append commits into target branch.
12. Count merged commits.
13. Display merge completion message.
14. End operation.

Pseudocode

```
ALGORITHM ProcessMerge()

BEGIN

    IF merge queue is empty THEN
        DISPLAY "No merge requests available"
        RETURN
    END IF

    Remove first merge request from queue

    source ← source branch
    target ← target branch

    Create empty set targetIDs

    FOR each commit in target branch DO

        Insert commit ID into targetIDs

    END FOR

    FOR each commit in source branch DO

        IF commit ID not in targetIDs THEN

            Add commit into temporary queue

        END IF

    END FOR

    WHILE temporary queue is not empty DO

        Remove commit from queue

        Append commit into target branch

    END WHILE

    DISPLAY "Merge completed successfully"
```

```
END
```

Time Complexity

T(n)=O(n)

The algorithm traverses commits from both source and target branches. The running time increases linearly with the number of commits.

6. Rollback Operation

The rollback operation restores a branch to a previous commit by deleting all commits after the selected commit.

Algorithm Steps

1. Start operation.
2. Search for the specified commit ID.
3. If commit is not found:
    - Display error message.
    - Stop operation.
4. Locate rollback commit node.
5. Delete all commits after selected node.
6. Update tail pointer.
7. Update branch size.
8. Display success message.
9. End operation.

Pseudocode

```
ALGORITHM Rollback(commitID)

BEGIN

    Search commitID in current branch

    IF commit not found THEN
        DISPLAY "Commit not found"
        RETURN
    END IF

    Delete all commits after target commit

    Update tail pointer
```

```
    Update branch size

    DISPLAY "Rollback completed successfully"

END
```

Time Complexity

T(n)=O(n)

The linked list may need to be traversed to locate the rollback point, resulting in linear complexity.

7. View Log Operation

Displays all branches together with their complete commit histories.

Algorithm Steps

1. Start operation.
2. Traverse all branches stored in branch table.
3. For each branch:
    - Display branch name.
    - Traverse commit linked list.
    - Display commit details.
4. Continue until all branches are displayed.
5. End operation.

Pseudocode

```
ALGORITHM ViewLog()

BEGIN

    FOR each branch in branch table DO

        DISPLAY branch name

        FOR each commit in branch DO

            DISPLAY commit details

        END FOR

    END FOR

END
```

Time Complexity

T(n)=O(n)

Every commit in every branch must be traversed and printed.

Overall Time Complexity

Operation	Data Structure Used	Time Complexity
Commit	Linked List	O(1)
Create Branch	Linked List Copy	O(n)
Switch Branch	Hash Map	O(1)
Enqueue Merge	Queue	O(1)
Process Merge	Queue + Set + Linked List	O(n)
Rollback	Linked List	O(n)
View Log	Linked List Traversal	O(n)